

MU5MAM30 - Projet2+4

Lingxi Song

March 2026

1 Project Overview

In this project, I attempted to complete **Project 2: Adding a boundary and boundary constraints**, and **Project 4: Elimination tree and direct sparse Cholesky solver** on my own. I have uploaded the current code to the GitHub repository, branch: `song`. In class, we completed the contents of multiple files. Here, I renamed the `navier_stokes.cpp` file to `navier_stokes_CG.cpp`, with the intention of distinguishing between boundary conditions and equation computations.

The final project is presented as a set of executable commands available in the terminal: `test_poisson`, `test_poisson_dirichlet`, `test_NS_CG`, `test_NS_CG_nonpenetrating`, `test_NS_CG_noslip`, and `test_NS_cholesky`. Among these, the three commands that introduce modified boundary conditions additionally support the half-sphere domain.

2 Project 2

This project requires us to perform simulations on a domain with boundaries. Next, I will explain what I have done from three aspects: **domain modification**, the addition of the **non-penetrating** condition, and the transform of the **no-slip** condition.

2.1 domain with boundaries

The originally used cube and sphere are both closed surfaces. Therefore, I added a `half_sphere` file (including `.h` and `.cpp`) to the `mesh` module, then I changed `makelist` file to add it into the instruction. The approach consists of taking the sphere mesh and retaining only the portion where $z \geq 0$, then assembling a new mesh from the remaining vertices and triangles. Since the sphere is derived from a subdivided cube, if we cut each side of the cube in even parts, the vertices will be projected onto the equator ($z = 0$), so no triangles are lost at the boundary when the lower hemisphere is deleted.

The addition of a new domain is only meant to illustrate the range of applicability. The main focus of this project is to apply the non-penetrating condition and, further, the no-slip condition into the original equation solver.

2.2 non-penetrating condition

In the original `project` files, it is already stated that the former imposes the boundary condition $\partial\Omega : \Psi = 0$, while the latter builds upon it by additionally enforcing $\partial\Omega : u = 0$ (which means $\partial\Omega : \Psi = 0$ and $u = 0$). Although both conditions introduce Dirichlet constraints during the boundary iteration, the use of the weak formulation together with $\omega = \text{curl } u$ requires special treatment of the condition $u = 0$, transforming it into a corresponding property in terms of ω .

2.2.1 about `set_zero_mean`

In code original, we solve:

$$\begin{cases} (M + \nu \Delta t S) \Omega^{m+1} = M \Omega^m + \Delta t \cdot T(\Omega^m, \Psi^m) \\ S \Psi^{m+1} = -M \Omega^m \end{cases}$$

Furthermore, since no boundary conditions were imposed in the original formulation, two additional fixes were required. First, to address the existence of a solution for ψ , we exploited the fact that S is symmetric positive definite when restricted to the orthogonal complement of its kernel (which has dimension 1, corresponding to constant functions); accordingly, `set_zero_mean(Momega.data)` was called inside `compute_stream_function`. Second, to ensure the uniqueness of the computed ψ and ω , `set_zero_mean(psi.data)` was applied after the conjugate-gradient solve in `compute_stream_function`, and `set_zero_mean(omega.data)` was applied inside `time_step`. With the introduction of boundary conditions, the operations for ψ are no longer necessary and have been removed, and I kept the operation for ω .

2.2.2 About $\partial\Omega : \Psi = 0$

This part is, in my view, the most important in the project. The criterion for identifying boundary vertices has already been described in the project files: a boundary edge is one that belongs to exactly one triangle. Using this criterion, I detect the presence of boundary vertices inside `NavierStokesSolver::NavierStokesSolver(const Mesh &m)`, and apply the non-penetrating condition when boundary vertices exist; otherwise, the original treatment is retained.

The key modification lies in `compute_stream_function`. My first instinct was to set $\Psi = 0$ on the boundary after solving, but I quickly realised that imposing the boundary condition as a post-processing step would produce overly sharp corrections at the boundary, polluting the values of interior vertices. The condition must therefore be incorporated into the solve itself.

This naturally requires modifying the matrices in the weak formulation. My first idea was to identify the boundary degrees of freedom of Ψ^{m+1} and, in the equation $S\Psi^{m+1} = -M\Omega^m$, zero out all off-diagonal entries in the rows and columns of S corresponding to boundary vertices, while setting the corresponding entries of $-M\Omega^m$ to zero. This preserves the structural properties of S and allows the system to be solved with CG. It also guarantees, by construction, that boundary values are zero and that boundary vertices do not influence the equations for interior vertices.

However, when the boundary values are non-zero, it is no longer possible to enforce the prescribed boundary values simply by zeroing the corresponding columns, since the contribution of the boundary degrees of freedom to the remaining equations must be retained. This would break the standard CG framework, so I use `projected_conjugate_gradient_solve` instead. The idea is analogous to directly modifying the weak-form matrices, but formulated by splitting the system into interior degrees of freedom I and boundary degrees of freedom B :

$$\begin{pmatrix} S_{II} & S_{IB} \\ S_{BI} & I \end{pmatrix} \begin{pmatrix} \Psi_I \\ \Psi_B \end{pmatrix} = \begin{pmatrix} -M_I\omega \\ 0 \end{pmatrix} \quad (1)$$

Since the boundary values are zero, it suffices to keep the search direction zero on the boundary and to force the residual to zero there as well. This ensures that only interior vertices are updated, while $\Psi_B = 0$ is maintained throughout.

By the same principle, I also wrote `poisson_non_homogeneous_Dirichlet.cpp`. Since this was the first file I modified, I did not use the ready-made PCG function, but instead wrote out the procedure explicitly.

2.3 Non-slip condition

The `compute_stream_function` part is identical to the non-penetrating case. The main modification concerns the equation solved inside `time_step`. Once we know how to construct the right-hand side, PCG can be applied directly.

We therefore need to express $\omega = \text{curl } u$ in terms of quantities we know on the boundary. To this end, I use Thom's Formula. Since $u = \nabla^\perp \Psi$, the no-slip condition $u = 0$ implies $\nabla \Psi = 0$ on $\partial\Omega$. Performing a Taylor expansion at a boundary vertex i towards an interior neighbour j gives:

$$\Psi_j \approx \underbrace{\Psi_i}_{=0} + \underbrace{\nabla \Psi|_i}_{=0} \cdot (x_j - x_i) + \frac{1}{2} \underbrace{\Delta \Psi|_i}_{=\omega_i} \|x_j - x_i\|^2 \quad (2)$$

Simplifying:

$$\Psi_j \approx \frac{1}{2} \omega_i \|x_j - x_i\|^2 \quad (3)$$

Solving for ω_i and averaging over all interior neighbours yields Thom's Formula:

$$\omega_i^{m+1} \Big|_{\partial\Omega} = \frac{1}{|\mathcal{N}_i|} \sum_{\substack{j \in \mathcal{N}_i \\ j \notin \partial\Omega}} \frac{2 \Psi_j^m}{\|x_i - x_j\|^2} \quad (4)$$

The time-step equation reads:

$$(M + \nu \Delta t S) \omega^{m+1} = \underbrace{M \omega^m + \Delta t T(\omega^m, \Psi^m)}_{=: b} \quad (5)$$

Setting $A = M + \nu \Delta t S$ and splitting into blocks:

$$\begin{pmatrix} A_{II} & A_{IB} \\ A_{BI} & A_{BB} \end{pmatrix} \begin{pmatrix} \omega_I^{m+1} \\ \omega_B^{m+1} \end{pmatrix} = \begin{pmatrix} b_I \\ b_B \end{pmatrix} \quad (6)$$

Since ω_B^{m+1} is already determined by Thom's Formula, the interior values ω_I^{m+1} can be obtained by PCG, with the boundary degrees of freedom held fixed throughout the iteration.

3 Project 4

Following the requirements of the `project` files, I studied the sparse elimination tree and Cholesky algorithms from the reference book, implementing Algorithms 4.2, 4.3, and 5.6. From the project's perspective, the modification consists simply of replacing the CG solver with the Cholesky factorization at the relevant steps; the main focus of this part therefore lies in the method itself.

3.1 Overview

The goal of Cholesky factorization is to decompose a SPD matrix A as:

$$A = LL^T \quad (7)$$

where L is a lower triangular matrix. Once L is known, solving $Ax = b$ reduces to two cheap triangular solves:

$$Ly = b \quad (\text{forward substitution}), \quad L^T x = y \quad (\text{backward substitution}) \quad (8)$$

In the context of the Navier-Stokes solver, A is factorized once before the time loop, and only the two substitutions are needed at each time step. This is the main advantage over CG: the factorization is more expensive upfront, but each subsequent solve is much cheaper.

The process is split into three stages, which must be executed in order.

3.2 Algorithm 4.2: Building the Elimination Tree

When performing Cholesky column by column, computing column j affects certain later columns. The elimination tree records this dependency: the parent of node j is defined as the first row $i > j$ such that $L_{ij} \neq 0$:

$$\text{parent}(j) = \min\{i > j : L_{ij} \neq 0\} \quad (9)$$

In plain terms: the elimination tree tells us *which columns must be computed before which*. If j is an ancestor of i in the tree, then column j must be finished before column i can be updated.

Since we do not yet know L , Algorithm 4.2 infers the tree directly from the sparsity pattern of A , using the fact that $A_{ij} \neq 0$ implies j is an ancestor of i . Path compression is used to avoid redundant traversals, in the same spirit as the Union-Find algorithm.

The output is a `parent[]` array. This is the only output of Algorithm 4.2, and it is passed directly to Algorithm 4.3.

3.3 Algorithm 4.3: Symbolic Analysis

After Cholesky, L may contain many more non-zero entries than A — these extra entries are called *fill-in*. Before computing any numerical values, we need to know *where* the non-zeros of L will be, so that memory can be allocated in advance. This is the role of Algorithm 4.3.

The key idea is: if row i of A has a non-zero at column k , then computing column k will affect row i . Furthermore, all ancestors of k in the elimination tree will also affect row i in turn. So the non-zero pattern of row i in L is:

$$\text{pattern}(i) = \{i\} \cup \bigcup_{\substack{k < i \\ A_{ik} \neq 0}} \{\text{ancestors of } k \text{ up to } i \text{ in the elimination tree}\} \quad (10)$$

In plain terms: for each row i , start from every non-zero entry k in that row of A , walk up the elimination tree using `parent[]`, and collect every node visited along the way. All these positions will be non-zero in L .

The output is the sparsity pattern of L — its `row_start` and `col` arrays — with memory allocated but no numerical values filled in yet. Algorithm 4.3 uses `parent[]` from Algorithm 4.2 at every step of the tree traversal; without it, the algorithm cannot run.

3.4 Algorithm 5.6: Numerical Factorization

With the memory layout of L established, Algorithm 5.6 fills in the actual numerical values. It processes columns left to right. For each column j :

Step 1 — compute the diagonal entry:

$$L_{jj} \leftarrow \sqrt{L_{jj}} \quad (11)$$

Step 2 — scale the sub-diagonal entries of column j :

$$L_{ij} \leftarrow \frac{L_{ij}}{L_{jj}} \quad \forall i \in \text{col_rows}(j) \quad (12)$$

Step 3 — rank-1 update of the trailing sub-matrix:

$$L_{ik} \leftarrow L_{ik} - L_{ij} \cdot L_{kj} \quad \forall k, i \in \text{col_rows}(j), i \geq k \quad (13)$$

where $\text{col_rows}(j)$ is the list of row indices $i > j$ such that $L_{ij} \neq 0$, precomputed from the pattern found by Algorithm 4.3. Each entry is located in memory using binary search (`find_ptr`), so no zero entries are ever touched.

3.5 Handling the Non-SPD Stiffness Matrix

In `navier_stokes_cholesky.cpp`, the stiffness matrix S is only *semi*-positive definite, not strictly SPD. This is because constant functions lie in its kernel:

$$\nabla c = 0 \implies S \cdot \mathbf{1} = 0 \quad (14)$$

so $\mathbf{1}$ is an eigenvector of S with eigenvalue zero. In the CG solver this was handled by `set_zero_mean`, which restricts the solution to the orthogonal complement of the kernel. However, Cholesky factorization requires strictly positive pivots at every step:

$$L_{jj} = \sqrt{L_{jj}} > 0 \quad (15)$$

A zero (or negative) pivot causes the factorization to fail immediately. The fix is to add a small diagonal shift:

$$S_{\text{shift}} = S + \epsilon I, \quad \epsilon = 10^{-6} \cdot \frac{\sum_i S_{ii}}{N} \quad (16)$$

This raises all eigenvalues by ϵ , turning the zero eigenvalue strictly positive while leaving the rest essentially unchanged. The shift is proportional to the average diagonal entry of S , so it scales with the matrix and remains negligible relative to the true eigenvalues. The error introduced in the solution is $O(\epsilon)$, which has no visible effect on the simulation.

In contrast, $A = M + \nu \Delta t S$ requires no such shift, because the mass matrix M is already strictly positive definite and can be factorized directly.

4 Problems that I've met

1. My Poisson solver appears to work correctly, but the Navier-Stokes solver diverges rapidly. I was unable to identify the source of the problem in my code.
2. For creating a new domain with boundary, I chose the hemisphere rather than a square grid, because the visualisation includes a z -axis and I did not know how to remove it from the plot.

3. When considering the modification of the weak-form matrices for the condition $\partial\Omega : \Psi = 0$, I wondered whether zeroing out certain matrix entries might prevent the influence of boundary vertices from propagating throughout the domain, or conversely make their influence disproportionately strong. Upon reflection, however, I concluded that this is not an issue: the only entries that are modified are the four off-diagonal entries immediately adjacent to each boundary vertex in the matrix, all other entries in those rows and columns being already zero. The boundary vertices therefore influence their immediate neighbours directly, and through them propagate their effect layer by layer into the interior of the domain. This behaviour is physically reasonable.
4. When addressing the non-SPD issue encountered in the Cholesky factorization, I chose to add a diagonal shift $S_{\text{shift}} = S + \epsilon I$ rather than modifying the algorithm itself. The main reason is pragmatic: this approach requires no structural changes to the factorization code, and since $S + \epsilon I$ shares exactly the same sparsity pattern as S , the symbolic phase (Algorithms 4.2 and 4.3) does not need to be rerun. I am aware, however, that this is not a perfectly rigorous solution — even though the perturbation is very small, it does alter the system being solved, and I cannot guarantee that the introduced error is always negligible in all configurations.

As for the choice of ϵ , I did not want to use a fixed absolute value such as 10^{-6} , because this would be poorly scaled: for a matrix whose diagonal entries are of order 10^6 , such a shift would be far too small to have any effect, while for a matrix with very small entries it might be unnecessarily large. Instead, I tied ϵ to the average diagonal entry of S :

$$\epsilon = 10^{-6} \cdot \frac{\sum_i S_{ii}}{N} \quad (17)$$

This makes the shift relative to the matrix’s own scale, so it adapts automatically to different mesh sizes and problem configurations. The factor 10^{-6} is a conservative engineering choice: large enough to push all pivots safely above zero, yet small enough that the perturbation remains negligible compared to the true non-zero eigenvalues of S .